

programs that demonstrate certain features and changes brought in by successive standards. But here comes an important question. What compiler are we going to use? There are many C++ compilers available depending on the operating system and the architecture being used. But from an open source software perspective the choices are obvious — g++ and clang++. The Clang compiler (invoked with the command *clang++*) from the LLVM compiler infrastructure project supports many of the latest features of C++. But we will use the very popular C++ compiler called g++, a part of GCC (GNU Compiler Collection), to compare the features offered and revoked by successive standards of C++. The choice not only depended on the popularity of g++ but also on the support for the latest features of C++. Here, again g++ was a winner over clang++. Of course there are areas where clang++ outperforms g++. But that discussion is for another day.

Before we proceed any further, we need to have an answer for the following question. What is the need for standardisation of programming languages? Imagine two software companies A and B designing compilers for C++. Company A decides to have 4 byte integers in its compiler. Without an accepted standard to adhere to, nothing stops company B from having a 40 byte integer in its compiler. And within no time, we will have two versions of C++ — C++A and C++B. So, it is always good to have standards on which compiler designers can rely. Now, let us begin our discussion of different C++ standards with the ‘Stroustrup Standard’ of C++.

The pre-98 ‘Bjarne Stroustrup’ standard

As mentioned earlier, Bjarne Stroustrup wanted to extend the C programming language, and by 1979 he was able to add features like classes, member functions, private and public access control, etc, to C. But soon after

the language got the attention of professional programmers, around 1984, ‘C with classes’ was renamed as C++. By 1985, C++ was available outside Bell Labs. In 1985, a C++ textbook written by Bjarne Stroustrup called ‘The C++ Programming Language’ was first published. It instantly became the *de facto* standard of C++ and remained so until 1998. By 1985, C++ had additional features like virtual functions and operator overloading. In 1990, Margaret A. Ellis and Bjarne Stroustrup published a C++ manual called ‘The Annotated C++ Reference Manual’. This further helped the standardisation of C++. According to the C++ manual of 1990, C++ only has 48 keywords. Cfront 1.0, Cfront 2.0 and Cfront 3.0 are three C++ compilers for this version of C++ that existed before 1998. GCC version 1.15.3, released in 1987, started supporting C++.

Unfortunately, programs written in the pre-98 version of C++ cannot be tested any more as neither g++ nor clang++ has an option to compile such C++ programs. Now consider the C++ program *pgm1.cc* given below. Notice that this short program does not produce any output.

```
#include<iostream>
int main( )
{
    struct str;
    struct str;
    return 0;
}
```

Both g++ and clang++ compile this program without any errors because only multiple definition of a variable causes an error in C++ (multiple declaration of a variable is fine). This rule is mentioned in the 1990 C++ manual. I believe this rule is a relic of the past and GCC being backward compatible to some extent is the only reason why such code is still tolerated. The compilation and execution steps of the program *pgm1.cc* using g++ as well

as clang++ are given below.

```
>> g++ pgm1.cc
>> ./a.out
>> clang++ pgm1.cc
>> ./a.out
```

C++98 standard

The need for standardisation increased along with the rise in the popularity of C++. In 1991, ISO (International Organisation for Standardisation) formed an ISO C++ committee for standardisation. This committee was called WG21, officially ISO/IEC JTC1 (Joint Technical Committee 1) / SC22 (Subcommittee 22) / WG21 (Working Group 21). The first official standard, informally called C++98 (official document ISO/IEC 14882:1998), was finalised and adopted in 1998. Figure 1 shows the logo of C++ adopted by ISO.



Figure 1: The logo of C++ adopted by ISO

Now, let us learn about the features of this first official C++ standard. Fifteen more keywords were added to the existing 48 keywords from the 1990 C++ manual, making the total number of keywords 63. The newly added keywords were *bool*, *true*, *false*, *const_cast*, *dynamic_cast*, *reinterpret_cast*, *static_cast*, *using*, *namespace*, *typeid*, *typename*, *wchar_t*, *mutable*, *explicit* and *export*. Further, there are 11 reserved alternative representations for certain operators and punctuators. They are *and*, *and_eq*, *bitand*, *bitor*, *compl*, *not*, *not_eq*, *or*, *or_eq*, *xor* and *xor_eq*. Notice that they remain reserved in every subsequent standard